JavaScript

Visual Analytics – Practical Part – P03

What is JavaScript?

- JavaScript is a cross-platform, object-oriented **scripting language**
- **Client-side** version runs in the browser (e.g. Chrome, Firefox, Edge, …)
- **Server-side** version runs in the OS, allowing more advanced features

JavaScript **IS NOT** Java! They have only similar syntax

Client-Side JavaScript

- Runs inside a browser (e.g., Chrome, Edge, Firefox, Safari, …)
- Extends the core language with functionalities that allows to interact with the web pages:
  - Modify the Document Object Model (DOM) i.e. the webpage
  - Respond to user events like clicks, text input, mouse movements

Server-Side JavaScript

- Runs on the operating systemExtends the core language with functionalities that allows to interact with the operating system:
  - Access to filesystem
  - Access to the network
  - …

A server-side JavaScript program is similar to a Python program:

- To be executed it needs a particular software installed, called JavaScript Engine. **Node.js** is one of them.

JavaScript Guides

We don’t need a textbook, most of the information are available online:

- Mozilla Developer Guide
  - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/
- The Modern JavaScript Tutorial
  - https://javascript.info/

Setup the developing environment



- Install **Node.js**, any version >= **22.x** is ok
  - https://nodejs.org



- **Goole Chrome** browser
  - Not all browser supports all the newest JavaScript features. Google Chrome is one of the most updated.



Client-side JavaScript

The HTML tag <script> … </script> allow to insert JavaScript code in the page

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>A webpage</title>
  </head>

  <body>

    <div id="root">
      <button id="btn1">Click me!</button>
    </div>

    <div id="content"></div>

    <script>

      function myListener() {
        const parent = document.getElementById("content")

        const p = document.createElement("p")
        p.textContent = "Hello World!"

        parent.appendChild(p)
      }

      let btn1 = document.getElementById("btn1")
      btn1.onclick = myListener


    </script>

  </body>
</html>
```

P03_CODE/01

```
    <script>

      function myListener() {
        const parent = document.getElementById("content")

        const p = document.createElement("p")
        p.textContent = "Hello World!"

        parent.appendChild(p)
      }

      let btn1 = document.getElementById("btn1")
      btn1.onclick = myListener


    </script>
```

Client-side JavaScript

The HTML tag <script> allow also to import the code from external files

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>A webpage</title>
  </head>

  <body>

    <div id="root">
      <button id="btn1">Click me!</button>
    </div>

    <div id="content"></div>

    <script src="myscript.js"></script>

  </body>
</html>
</html>
```

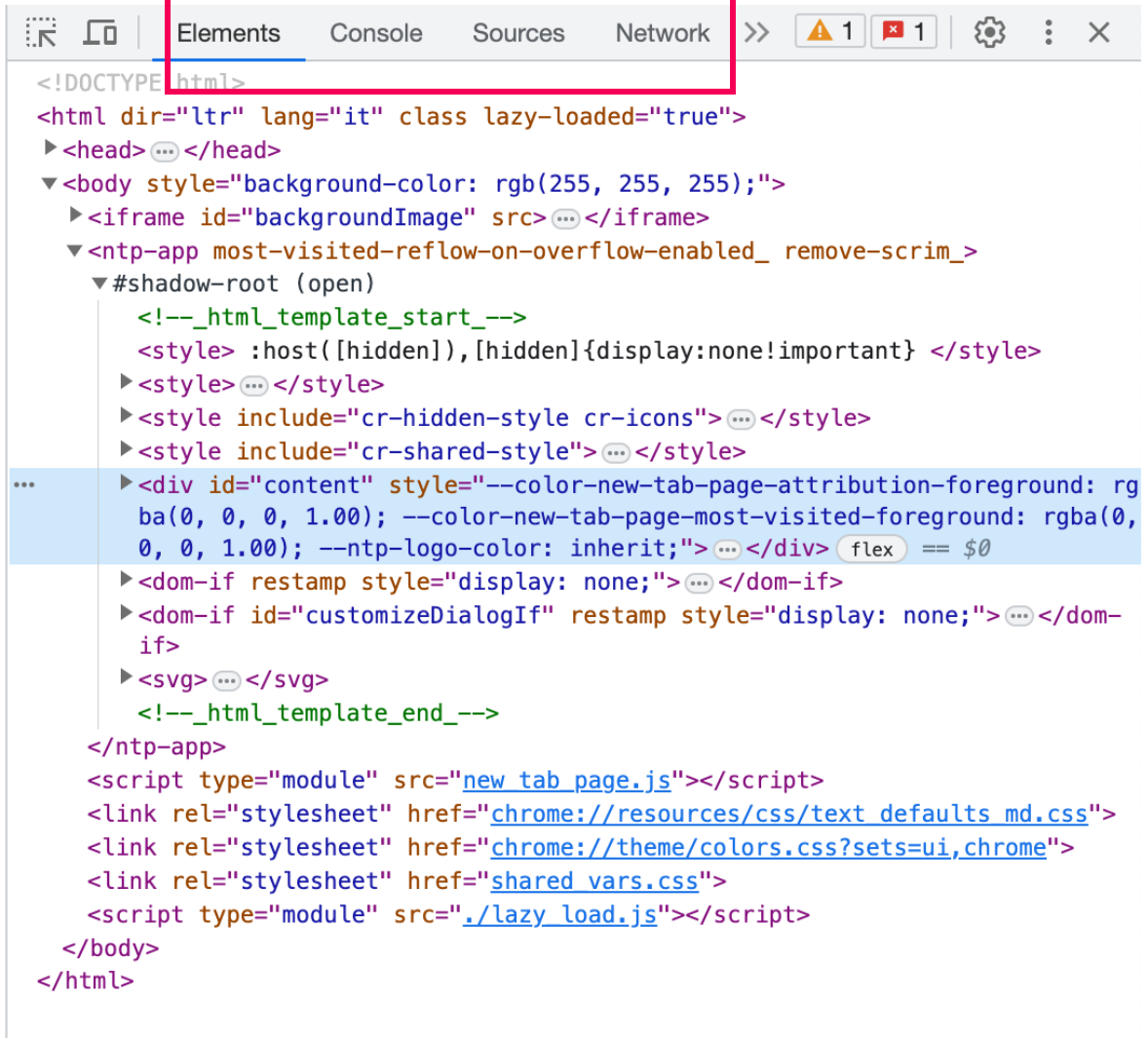
P03_CODE/02

The code is loaded in the same place in which is imported

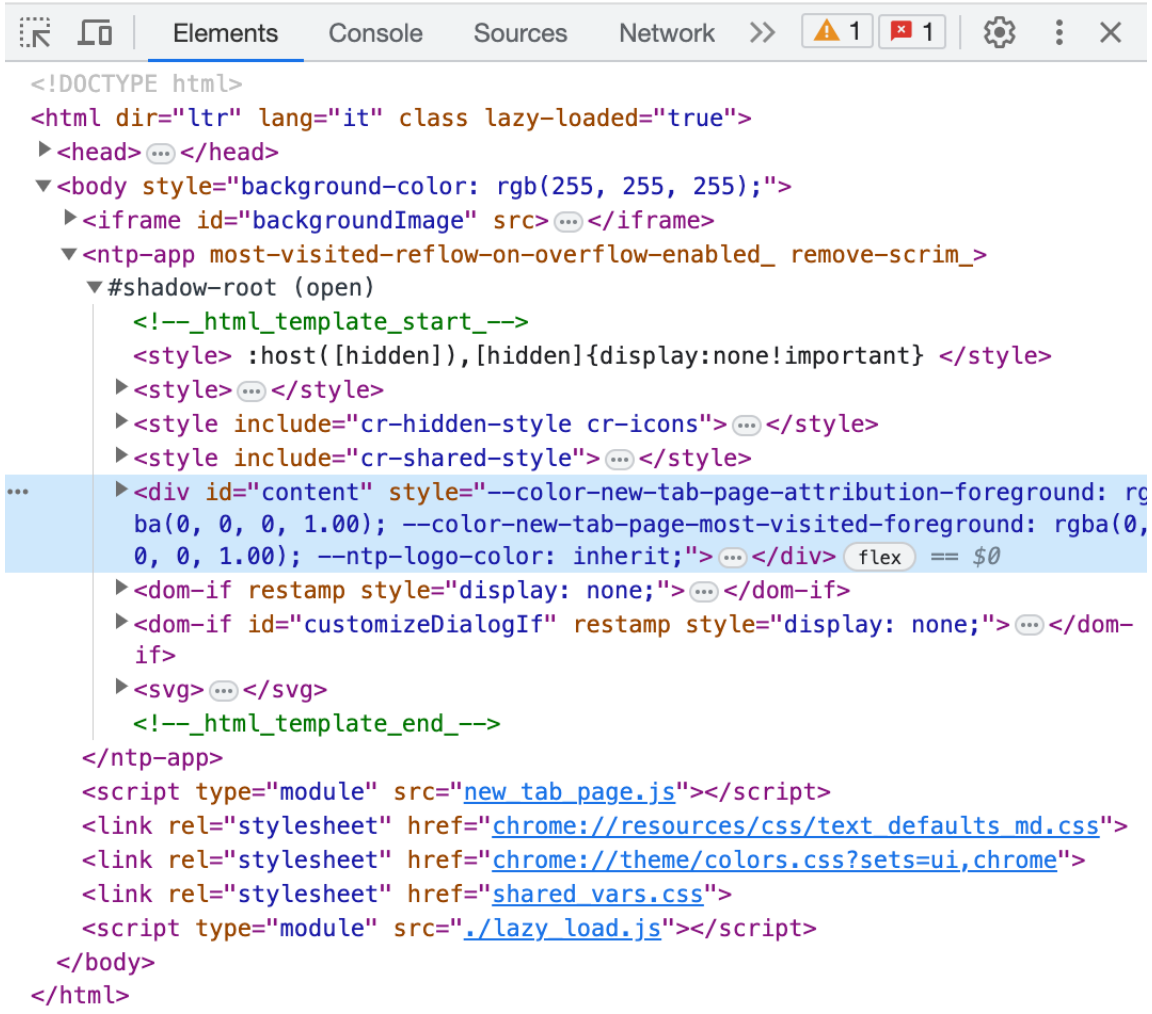
JavaScript Console on Google Chrome

- The browser provide also a console to insert JavaScript code and execute it on the fly

- Right click on the page → Inspect

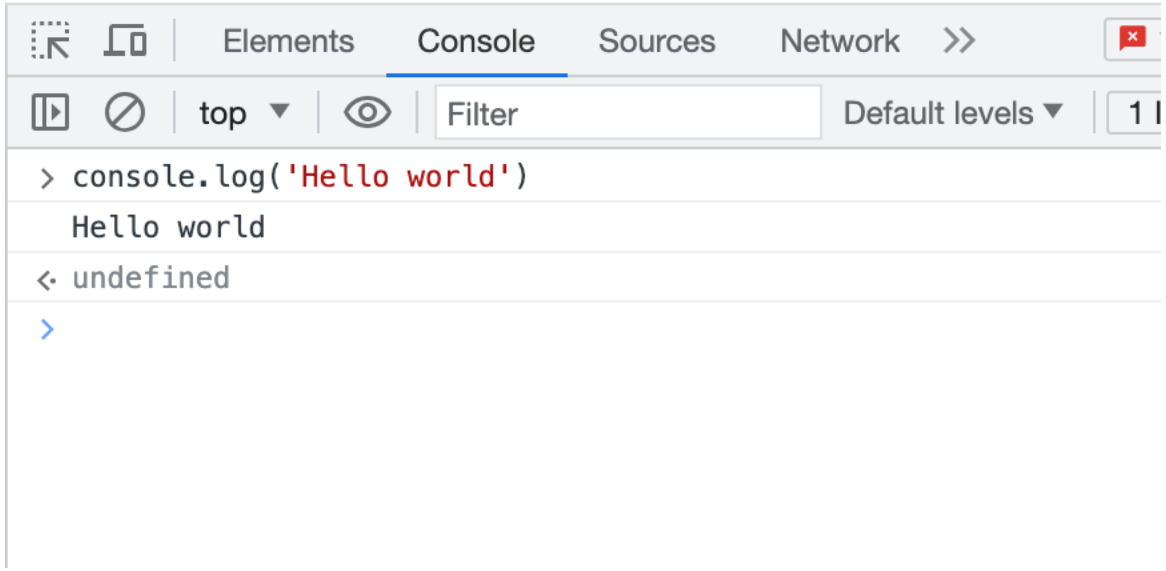


Elements



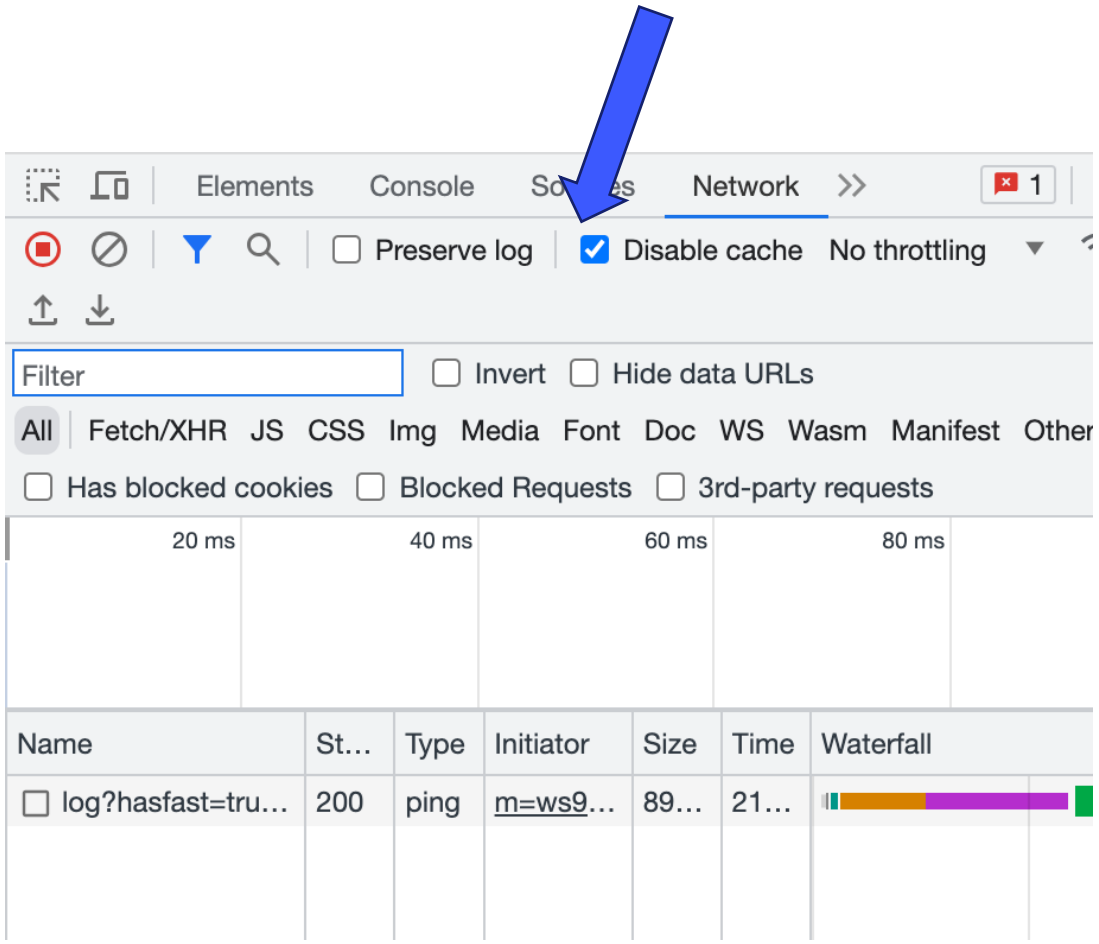
The Elements tab show the elements composing the html page. It allows to modify properties and attributes

Console



The Console tab allow to manually insert JavaScript code in the page

Network



The Network tab show information about the network, downloaded files etc.

Is important to active the **Disable Cache** checkbox

In general, browser tends to memorize in cache the scripts. When we modify the script are not always updated

JavaScript Tutorial

Introduction to the basic notions

The use of comments

```
/*

This is a
multiple line comment,
like Java, C, etc

*/


// This is a single line comment

```

The use of semicolon ;

Semicolons ; are NOT MANDATORY line breaks work as the same

```
1 /* This is an Hello World example from the file example-01.js */
2
3 // a semicolon ends the instruction
4 var a = 10;
5
6 // the semicolon is NOT mandatory, because there is a single instruction on the line, this does
  not produce an error
7 var b = 5
8
9 var c = 'a string'; var d = 8; // a semicolon here breaks two instruction on the same line
10
11
12 console.log(a + b)
13 console.log(c); console.log('Variable d is ' + d)
```

Hello World

```
1 /* This is a function that prints an Hello World message */
2 function helloWorld () {
3   const str = 'Hello World from JavaScript!'
4   console.log(str)
5 }
6
7 /* Now we call the function */
8 helloWorld()
9
```

Variable declarations

- JavaScript has **three** kinds for variable declarations:
  - `var` → Declares a variable, optionally initializing it to a value
  - `let` → Declares a **block-scoped**, local variable, optionally initializing it to a value
  - `const` → Declares a **block-scoped**, read-only named constant

```
1 // this is a generic variable
2 var myVariable = 10
3
4 // this is a constant and cannot be ressigned
5 const myConstant = 42
6
7 // this is a variable that can be reassigned
8 let anotherVariable = 5
9
```

The VAR variable

```
1 var n = 10
2 var tot = 0
3
4 for(var i=0; i<n; i++) {
5   tot += i
6 }
7
8 console.log(tot) //-> 45
9
10 /* we can read/write variable i */
11 i += 1
12 console.log(i) //-> 11
13
14 /* ERROR -> variable i
15  has been already declared */
16 var i = 'hello'
```

var is globally visible, we can read/write it in any following part of the code, after its declaration

We cannot redeclare a variable var

AVOID to use var, only if you really need its behavior. The fact that is globally visible can lead to programming mistakes

USE the let variable instead

The LET variable

```
1 let n = 10
2 let tot = 0
3
4 for(let i=0; i<n; i++) {
5   tot += i
6 }
7
8 console.log(tot) //-> 45
9
10 /* we CANNOT read/write variable i
11 because it is block-scoped */
12 i += 1 //-> ERROR
13 console.log(i) //-> ERROR
14
```


A let variable is block-scoped, i.e. can be accessed only inside the block in which is declared.

A block is a construction enclosed by parentheses: FOR, WHILE, IF, ELSE, …

Always use let instead of var, when it is possible.
This is a best practice that helps to limit mistakes.

The CONST variable

A const variable is block-scoped and read-only

```
1 const n = 10
2 let tot = 0
3
4 for(let i=0; i<n; i++) {
5   tot += i
6 }
7
8 console.log(tot) //-> 45
9
```



Use const if you don't have to modify its value

Special values: null and undefined

```
1 const x = null
2
3 const y = undefined
4
5
6 if(x != y){
7   console.log('x and y are NOT equal')
8 }
9
```

A null value is always assigned in the code

The undefined value is automatically setted if a value is missing, for example if a function has not a return statement, it returns undefined by default. Or if a function takes in input a parameter, and it is not passed when the function is called, the parameter is setted to undefined.

Display variables

```
1 const x = 10 * 10
2
3
4 console.log(x) // to print a basic message, like print(x) of Python
5
6
7 console.warn(x) // to print a warning message
8
9
10 console.err(x) // to print a error message
11
12
```

Strictly Equality

- The strictly equality/disequality checks both the value and the type of the variable

```
1 const a = 100
2 const b = '100'
3
4 /* EQUALITY */
5 console.log(x == y) //-> true
6
7 /* STRICT EQUALITY */
8 console.log(x === y) //-> false
9
```

```
a === b // strictly equal
a !== b // strictly not equal
```

```
null == undefined //--> true
null === undefined //--> false
```

Strings

Strings can be declared with different types of quotes

```
1 const a ='alpha' // single quotes
2
3 const b = "beta" // double quotes
4
5 const c = `gamma` // grave accents aka backtick
6
```

Template Strings

Backticks allows to create strings with embedded expressions, by using the construct ${ … } Similar to python f-strings

```
1 const a = 3
2 const b = 2
3
4 const str = `a plus b is ${a+b}` //-> 'a plus b is 5'
```

Converting strings to number

```
1 const str = '10.5'
2
3 const x = parseInt(str) //-> 10
4
5 const y = parseFloat(str) //-> 10.5
6
```

Functions 1/2

```
1 function showMessage() {
2   let message = 'Hello, I am JavaScript!'; // local variable
3
4   console.log(message)
5 }
6
7 showMessage(); // Hello, I am JavaScript!
8
```

Function 2/2

```
let userName = 'John' // the function has access to outer variables

function showMessage() {
  let message = 'Hello, ' + userName
  console.log(message)
}

showMessage() // Hello, John

```

Arrow Functions

```
1 /* arrow function that sums two number*/
2 const sum = (a, b) => a + b
3
4 /* equivalent form as classic function */
5 function sum2(a, b){
6   return a + b
7 }
8
9 /* if we need more than expression in the function body, we need to use { } */
10 const sum3 = (a, b) => {
11   if(a > 0) return a + b
12   else return b
13 }
14
```

Arrays https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

- Arrays can be declared in two ways:
  - with squared parentheses (like Python)
  - instantiating the Array class, passing a length and an initial fill value that is copied in each element of the array

```
1  const myArray1 = ['a', 'b', 10, 23]
2
3  // creates an empty array of length 5
4  const myArray2 = new Array(5)
5
6  // creates empty array of length 5, and set all the elements equal to 'a'
7  // fill() is a method of the class array
8  const myArray3 = new Array(5).fill('a')
9
10
11 console.log(myArray1.length) // return the length of the array
12
```

Useful methods of the class Array

```
/* we create an array */
let myArr = [10, 20, 30]

/* THE map method transform each element of the array
returning a new array */

/* newArray is [1, 2, 3] */
let newArray = myArr.map((v, i) => {
  // this function is called for each element of the array
  // it must return a value

  // i is the index of the current element
  // v is the value of myArray[i]

  return v / 10
})


/* THE forEach method pass to a function each element of the array */
myArr.forEach((v, i) => {
  console.log(`The element at index ${i} is ${v[i]}`)
})
// The element at index 0 is 10
// The element at index 1 is 20
// The element at index 2 is 30

```

Useful methods of the class Array

https://javascript.info/array-methods

```
// adds item to the end
arr.push(item)

 // extracts an item from the end
arr.pop()

// extracts an item from the beginning
arr.shift()

// adds item to the beginning
arr.unshift(item)

```

```
/* we create an array */
let myArr = [10, 20, 30]

/* THE map method transform each element of the array
returning a new array */

/* newArray is [1, 2, 3] */
let newArray = myArr.map((v, i) => {
  // this function is called for each element of the array
  // it must return a value

  // i is the index of the current element
  // v is the value of myArray[i]

  return v / 10
})


/* THE forEach method pass to a function each element of the array */
myArr.forEach((v, i) => {
  console.log(`The element at index ${i} is ${v[i]}`)
})
// The element at index 0 is 10
// The element at index 1 is 20
// The element at index 2 is 30

```

Dictionary (called Object)

- Dictionary can be declared like Python
- Dictionary keys (a, b, c, … in the example) can be also written without quotes IF the key is not a number or it does not contain spaces

```
1 const myDict = {
2     a: 10,
3    'b': 20,
4    'c': 'hello',
5    'a complex key with spaces': 1,
6     d: false
7 }
8
```

Iterate over a dictionary

```
let myDict = {
  'a': 10,
  'b': 20,
  'c': 30,
  'd': 40
}

const keys = Object.keys(myDict) // list of the keys [a,b,c,d]

keys.forEach(k => {
  console.log(`At key ${k} there is the value ${myDict[k]}`)
})
```

Set https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set

A set is a collection of unordered and unique objects

```
1 const s = new Set(['a', 'b', 10, 23])
2
3 s.add(5) // add an element
4
5 s.remove(10) // remove an element
6
7 console.log(s.size) // print the size of the set
8
```

Class basic syntax

```
1 class User {
2   constructor (name, surname, birthYear) {
3     this.name = name
4     this.surname = surname
5     this.birthYear = birthYear
6   }
7
8   getCompleteName () {
9     return this.name + ' ' + this.surname
10  }
11
12  getCurrentAge () {
13    const today = new Date()
14    const age = this.birthYear - today.getFullYear()
15
16    return age
17  }
18 }
19
```

```
1 const u = new User('John', 'Smith', 1950)
2
3 console.log(u.name) // 'John'
4 console.log(u.surname) // 'Smith'
5 console.log(u.getCompleteName()) // 'John Smith'
6 console.log(u.getCurrentAge()) // 73
7
```

Class inheritance

```
1 class Car {
2   constructor (brand, model, year) {
3     this.brand = brand
4     this.model = model
5     this.year = year
6   }
7 }
8
9 class Tesla extends Car {
10   constructor (model, year) {
11     super('Tesla', model, year)
12   }
13 }
14
```

Error handling

- Usually, a script “dies” (immediately stops) in case of an error, printing it to console.

- But there’s a syntax construct try...catch that allows us to “catch” errors so the script can, instead of dying, do something more reasonable

- Learn more at https://javascript.info/error-handling

```
try {

  // code...

} catch (err) {

  // error handling

}
```

Server-side JavaScript

Server-Side JavaScript

- Runs on the operating system, needs an engine (e.g. Node.js) to be executed
- Extends the core language with functionalities that allows to interact with the operating system:
  - Access to filesystem
  - Access to the network
  - …

A server-side JavaScript program is similar to a Python program:

- To be executed it needs a particular software installed, called JavaScript Engine. **Node.js** is one of them.

Node.js

- Cross-platform JavaScript runtime environment
- Based on the V8 JavaScript Engine, the core of Google Chrome

https://nodejs.org



Setup the developing environment



- Any text editor is ok, but for the course is suggested **Visual Studio Code**
  - https://code.visualstudio.com/

- Install **Node.js**, any version **>= 22** is ok
  - https://nodejs.org



How to run a server-side JavaScript program

- A JavaScript program is a text file with extension `.js`
- Suppose the file is `myprogram.js`
- In the console execute the command: `node myprogram.js`

NPM Package Manager https://www.npmjs.com/

- To install external libraries (package) there is `npm`
- Similar to `pip` for Python

- Two main commands:
  - `npm init` → create the project
  - `npm install foo` → install the package (i.e. library) called foo
  - `npm install --save-dev foo` → install the package as developing dependence meaning that is required for the development process, but is not required in the application that we are going to develop

Project Structure

- Inside a folder in which npm has installed some packages, we find:
  - **node_modules** → a folder containing all the files of the external packages, DO NOT EDIT files inside this folder, if you are on a git project, add the folder to gitignore
  - **package.json** → a json file specifying all the required packages and other information. Executing only `npm install` without the name of the package, inside a folder in which there is a **package.json** file, each dependency listed in the file will be installed.
  - **package-lock.json** → a json file in which are reported all information about dependency versions. Do not edit this file. It is automatic generated.

package.json



Hot to run a project

`npm start` to run the project that contains a package.json file

- It executes the script index.js
- Other scripts can be executed, in that cases, they must be specified in a filed of the package.json file

StandardJS

- Visual Studio Code extension for JavaScript Standard Style with automatic fixing.
- Helps to maintain code clear, improve readability, limits errors

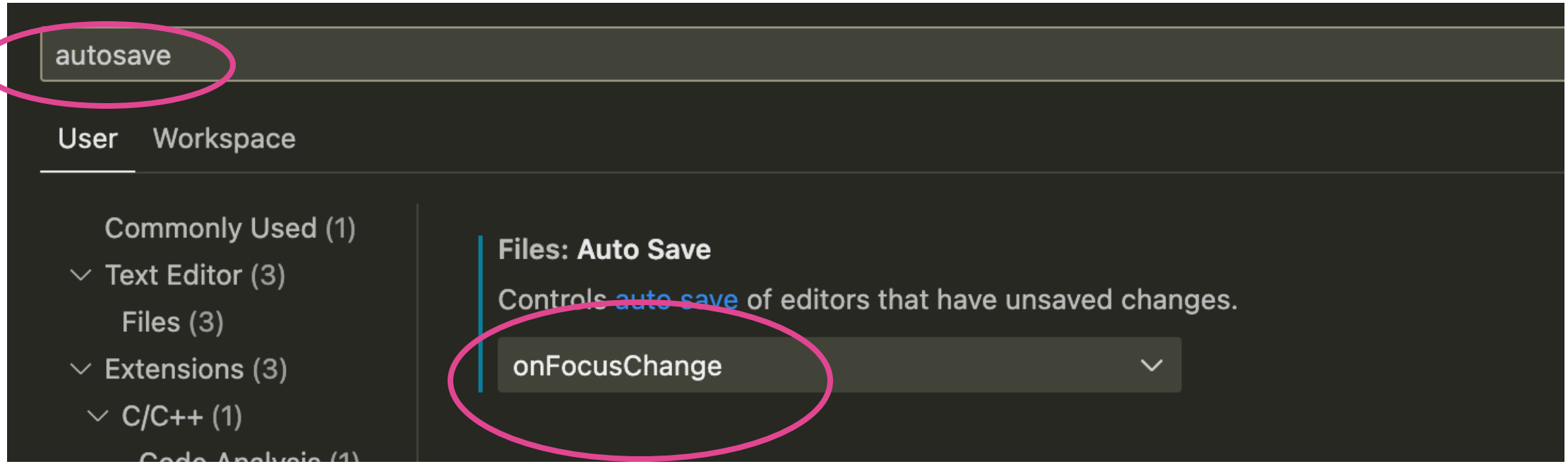


https://marketplace.visualstudio.com/items?itemName=standard.vscode-standard

Setup StandardJS

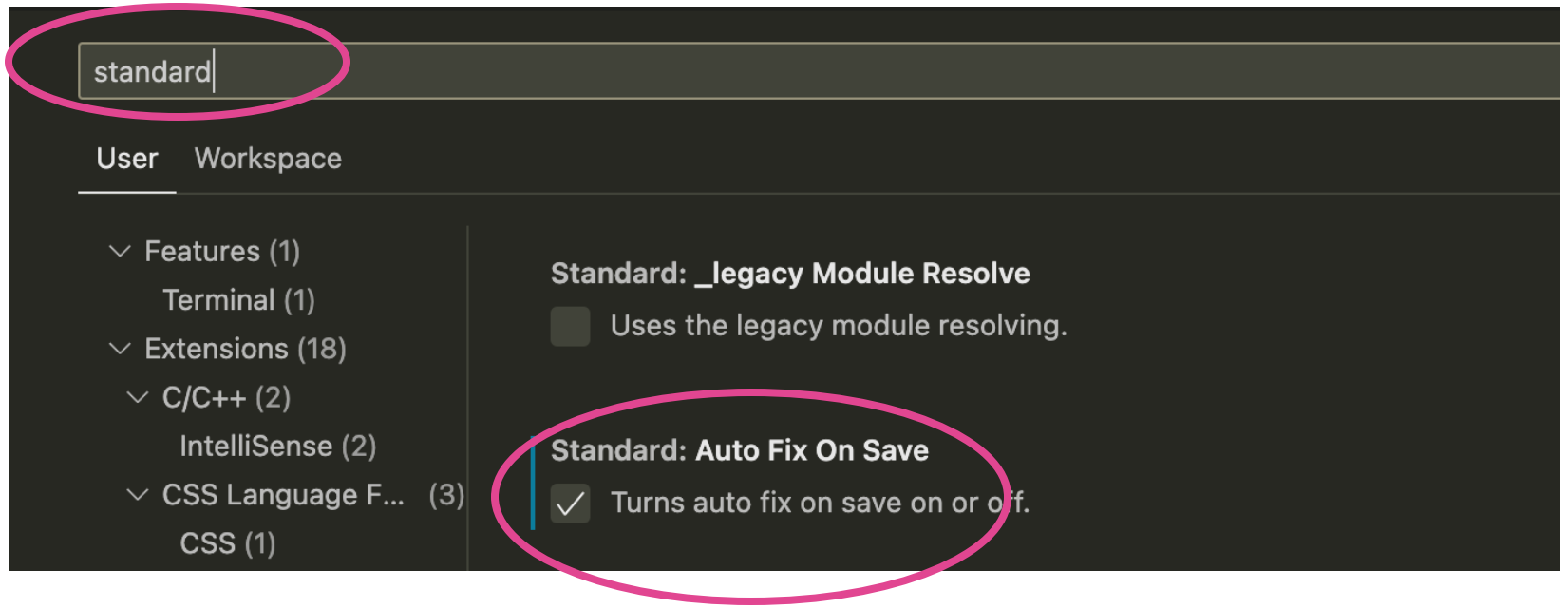
- Open VisualStudio code settings (StandardJS extesion has to be yet installed) https://code.visualstudio.com/docs/getstarted/settings

- Search for "autosave" and set to "**onFocusChange**"



Setup StandardJS

- Search for "standard" and enable "Auto Fix On Save"



`npm install --save-dev standard`

To use the standard.js extension:

- Configure the extension as specified in the previous slides
- In each project, must be installed as **development dependency**

EXAMPLE-03

A simple http server that responds with hello world.

Code available in P03_CODE/03

EXAMPLE-03

A simple http server that responds with hello world

```json
{
  "name": "code_01",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "standard": "^17.1.0"
  }
}
```

package.json

```js
const express = require('express')

const port = 1234

// create the server object
const server = express()

// set an endpoint
server.get('/', (req, res) => {
  res.send('Hello World!')
})

// start the server
server.listen(port, () => {
  // this is a callback called when the server is ready
  console.log(`Server active on port ${port}`)
})

```

index.js

EXAMPLE-03 | package.json

```
npm init
npm install express
npm install --save-dev standard
```

Express is a minimalist web server https://expressjs.com/
(Flask is something similar for Python)

```
{
  "name": "code_01",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "standard": "^17.1.0"
  }
}
```

Remember to add manually the scripts part

EXAMPLE-03 | index.js

```
1  const express = require('express')
2
3  const port = 1234
4
5  // create the server object
6  const server = express()
7
8  // set an endpoint
9  server.get('/', (req, res) => {
10   res.send('Hello World!')
11 })
12
13 // start the server
14 server.listen(port, () => {
15   // this is a callback called when the server is ready
16   console.log(`Server active on port ${port}`)
17 })
18
```

External modules are imported with the `require` command. Not only for libraries, but also for object from other files of the project.

Instantiate the server object

Set the function to be called when arrives a request to the endpoint '/'

Start the server, specify a callback function that is called when the server is ready

Import/Export on Server-side

- Use `module.exports = …` to export an object/variable/function/class

- Use `const foo = require('foo')` to import from the module foo installed as dependency

- Use `const foo = require('./path/foo')` to import from the module local module foo

webpack

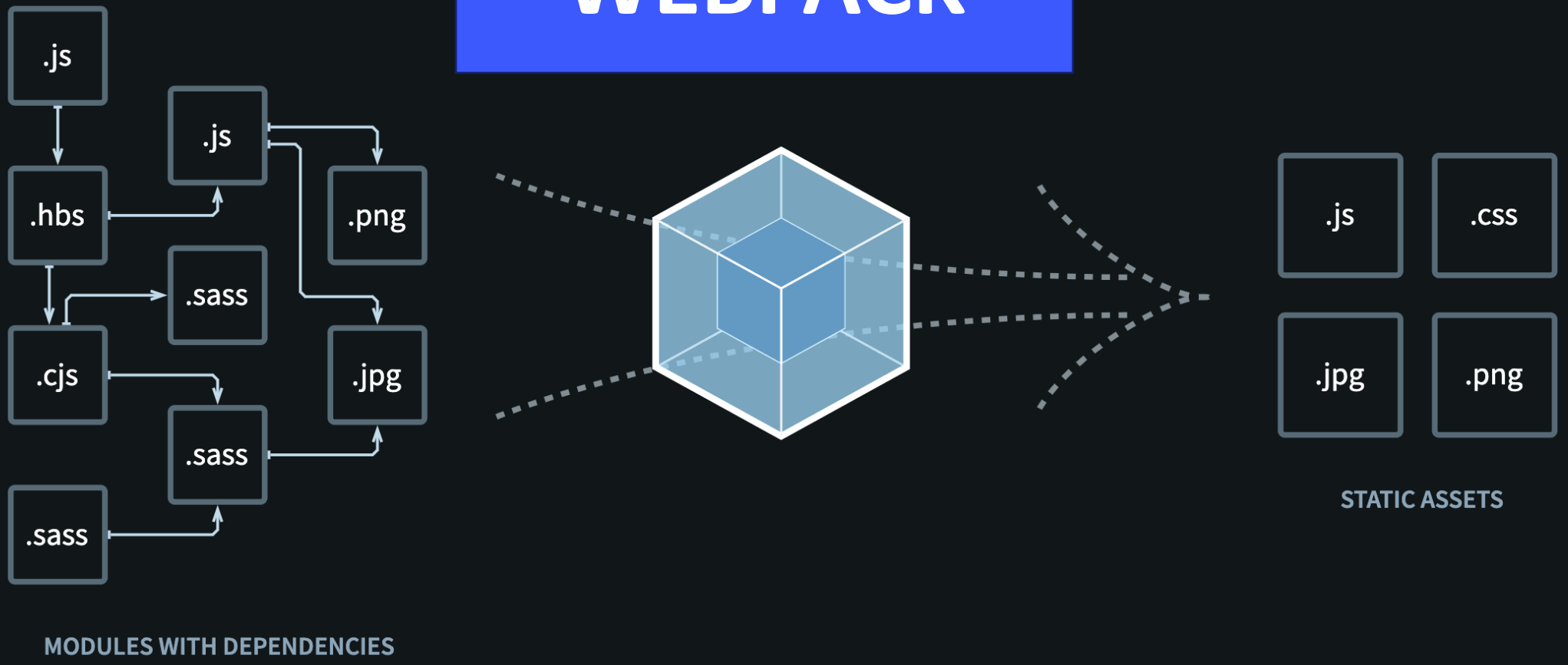


webpack

- **webpack** is a static module bundler for modern JavaScript applications
- Used to compile JavaScript modules: collect all dynamic dependencies and generate static files

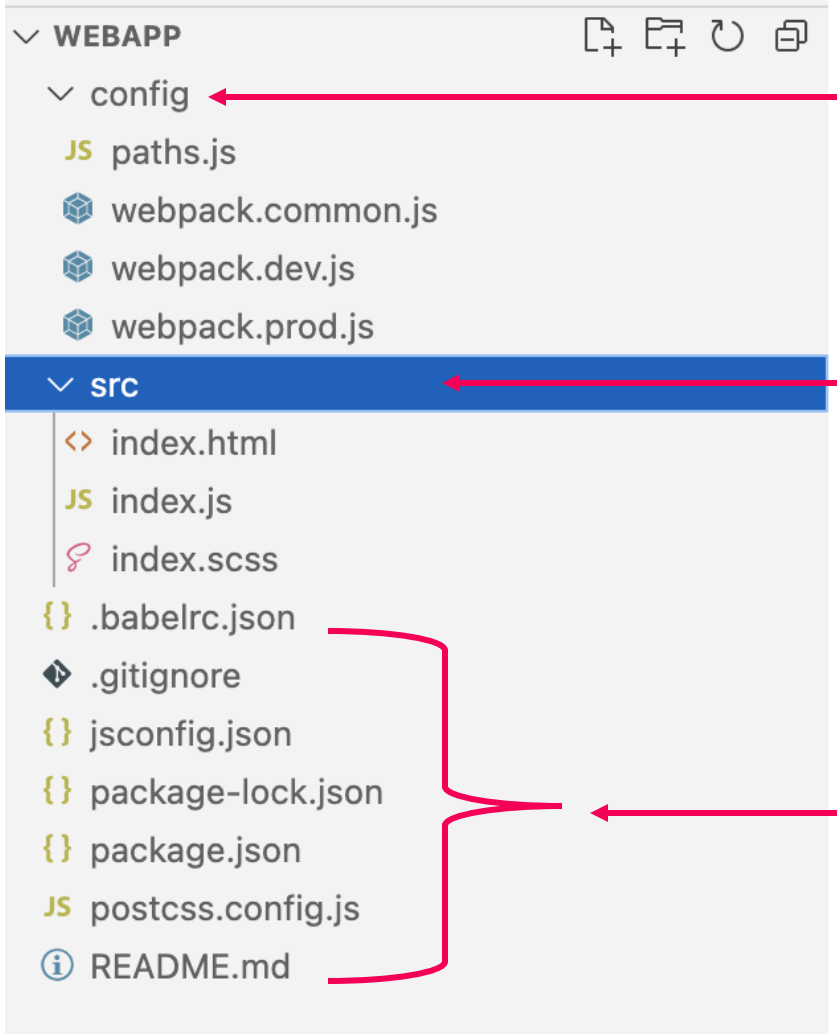


WEBPACK



https://webpack.js.org/

WebApp Template



In folder config webpack configuration files

In folder src source files of my application

- index.html
- index.css
- index.js
- …
- All the other scripts that I need

Configuration files of the project

Import/Export on Client-side

- Use `export` keyword to export an object/variable/function/class
- Use `export default` keyword to set the default export

- Use `import foo from bar` to import the default exported object
- Use `import {fn1, fn2} from bar` to import some objects